TriGCR: Constraint-Aware KG-Trie Reasoning for Knowledge-Grounded LLMs: DVI, GraphLite, and Embedding-Guided Path Selection

AIAA 4051 Final Research Project: Knowledge Grounded LLM Reasoner
Team Members: kaigeliang, fangzhehuang, and yaoxianshan
Project Manager: Jiujiu Chen

May 2026

Abstract

Large language models can answer complex questions, but their reasoning traces may contain unsupported facts. Graph-Constrained Reasoning (GCR) improves faithfulness by building a KG-Trie over knowledge-graph paths and constraining a KG-specialized LLM to generate only graph-valid reasoning paths. This project reproduces GCR, analyzes its failures, and implements three modifications to the graph-constrained reasoning pipeline: Decompose-Verify-Intersect (DVI), GraphLite, and embedding-guided KG-Trie construction. DVI adds a constraint-state controller: a general LLM decomposes a question into atomic constraints, a KG verifier collects per-constraint candidate sets, Python computes candidate intersections, and a confidence gate routes back to the GCR baseline when DVI evidence is unreliable. GraphLite replaces expensive KG-LLM verification with entity-level graph enumeration: relation selection, candidate-entity scoring, and selective two-hop expansion. Embedding-guided KG-Trie ranks candidate paths by question-path semantic similarity before trie construction and extends the path budget to preserve useful 3-hop evidence. On full RoG-CWQ, reproduced GCR reaches 54.55 F1 with Llama-3.1-8B, 54.50 F1 with Llama-2-7B, and 42.31 F1 with Qwen2-0.5B. Raw DVI alone underperforms the strongest baselines, but exact-size routed DVI improves Llama-3.1 from 54.55 to 54.63 F1 and Llama-2 from 54.50 to 54.51 F1, with oracle routing showing much larger headroom of 61.51 F1. GraphLite is substantially faster than KG-LLM DVI (3.09s vs. 12.72s per example) but lower quality (42.14 F1). Embedding-guided KG-Trie is close to Qwen GCR on full 2-hop evaluation and improves controlled 3-hop probes when paired with candidate reranking. The central conclusion is that KG path validity is necessary but not sufficient: complex KGQA needs explicit state for constraint satisfaction, answer typing, intersection, and temporal or numeric operators. The code and reproducibility artifacts are open source at https://github.com/kaigeliang/TriGCR.

1 Introduction

Knowledge graphs (KGs) store factual information as triples (h, r, t) between entities and relation types. They are therefore a natural grounding source for knowledge-intensive question answering. A free-form LLM can generate plausible but false reasoning chains, while a KG-grounded system can restrict the model to evidence that exists in the graph.

The baseline for this project is Graph-Constrained Reasoning (GCR) [8]. GCR retrieves KG paths around question topic entities, serializes them into a prefix trie, and constrains a KG-specialized LLM so that generated reasoning paths follow graph-valid continuations. A final general LLM then reads those paths and produces answers. This design is strong because it attacks path hallucination directly.

The weakness is that a prefix trie represents linear path validity, not full constraint satisfaction. ComplexWebQuestions (CWQ) contains composition, conjunction, comparative, and superlative questions. Many examples require intersecting evidence from multiple entities, checking an answer type, applying a date or numeric filter, or choosing an argmax/argmin. A path can be perfectly KG-valid while still ending at a mediator node, ignoring a temporal predicate, or satisfying only one branch of a conjunction.

We study the following question: can KG-Trie reasoning be extended from local path validity to explicit multi-constraint answer satisfaction? We answer this through three implemented modifications:

- **DVI**: decompose the question into atomic constraints, verify each constraint against the KG, intersect candidate sets programmatically, and route selectively between DVI and the original GCR output.
- **GraphLite**: replace token-level KG-LLM verification with explicit relation/entity enumeration, DeBERTa cross-encoder scoring, and selective two-hop candidate expansion, giving an efficiency-oriented verifier for DVI.
- **Embedding-guided KG-Trie**: rank paths by question-path semantic similarity before trie construction, allowing 3-hop evidence to be added without fully exposing the model to path explosion.

The project requirements ask us to reproduce the baseline, explore more than two KG-specialized LLMs, analyze failures by query type, and modify the graph-constrained decoding module for complex reasoning constraints. This

expanded technical report is organized around those requirements but is no longer page-limited: it includes the reasoning motivation, detailed algorithms, implementation decisions, experiment protocols, ablations, failure analysis, and reproduction notes.

What changed relative to the baseline. The original GCR pipeline has two stages: a KG-specialized LLM generates constrained paths, and a general LLM turns those paths into answers. Our changes keep this overall interface but modify the evidence construction and verification layer. DVI inserts a controller around GCR and makes candidate-set intersection explicit. GraphLite keeps the DVI controller but swaps the verifier for explicit relation/entity enumeration and neural scoring. Embedding-guided KG-Trie keeps the baseline decoder but changes which paths enter the trie. These are deliberately complementary: one changes aggregation, one changes verification cost, and one changes retrieval coverage.

Why three methods are useful. The archive shows that no single modification dominates. DVI is closest to a reasoning-schema change and provides the best evidence for constraint-aware reasoning, but it needs a reliable router. GraphLite is faster and more inspectable, but weaker as a final QA system. Embedding-guided KG-Trie improves small 3-hop probes but not the full 2-hop Qwen baseline by itself. Reporting all three methods is therefore more honest than presenting one cherry-picked result: together they identify where the baseline fails and which parts of the solution are already working.

2 Research Questions and Design Principles

This project started from a practical observation in the archived experiments: many incorrect answers are not arbitrary hallucinations. They are often supported by KG-valid paths that are locally plausible but globally insufficient. This leads to three research questions.

RQ1: Can graph-constrained decoding be made constraint-aware? GCR ensures that generated path tokens match a KG path prefix, but complex KGQA questions require more than prefix validity. The system must know whether the candidate answer satisfies a relation constraint, an attribute constraint, a type requirement, and sometimes a temporal or numeric filter. DVI addresses this by adding a controller that tracks constraint-level candidate sets and performs deterministic intersection.

RQ2: Can verification be made cheaper without losing all graph grounding? The KG-specialized LLM is expensive because it decodes token by token under a trie. GraphLite asks whether explicit graph enumeration plus learned scoring can replace this generation step for some examples. The integrated PathScorer is a more conservative version: it enumerates DFS paths and ranks them with a bi-encoder followed by a cross-encoder.

RQ3: Can the trie include longer evidence without uncontrolled path explosion? Many CWQ examples require bridge paths that are not present in a small 2-hop neighborhood. Naively increasing the hop limit creates many irrelevant paths and weakens constrained decoding. Embedding-guided KG-Trie construction uses semantic path ranking to decide which longer paths are admitted.

The design principles follow from these questions. The first principle is *explicit state*: intermediate objects such as constraints, path pools, candidate sets, intersection sizes, fallback flags, and timing measurements should be saved rather than hidden inside a final answer string. The second principle is *conservative integration*: because each modification can hurt some examples, the strongest reported improvement should use a simple, inspectable gate rather than a cherry-picked selection. The third principle is *diagnostic evaluation*: a lower end-to-end score can still be useful when it isolates a speed or coverage tradeoff that the final integrated system can exploit.

These principles explain why the report contains both positive and negative results. Raw DVI underperforms strong GCR baselines, but it supplies complementary answers that an oracle can exploit. GraphLite is faster but weaker, yet its path-reranker metrics show that supervised scoring improves path selection. Embedding-guided KG-Trie does not beat the full 2-hop Qwen baseline by itself, but controlled 3-hop probes show that semantic ranking can make longer evidence useful.

3 Related Work

Knowledge graph question answering. Early KGQA systems map questions to semantic parses or logical forms over Freebase-style graphs [1, 21]. CWQ was built by composing WebQuestions into more complex forms, making conjunction, comparison, and superlative reasoning important [14]. These operators are exactly where a single path prefix becomes insufficient.

Graph-grounded LLM reasoning. Retrieval-and-reasoning systems such as KV-MemNN, GraftNet, PullNet, and retrieval-augmented generation retrieve facts or subgraphs before answer prediction [9, 11, 12, 6]. Recent LLM-era methods guide graph search with language models, including RoG and Think-on-Graph [7, 13]. GCR differs by constraining generation itself: the KG-specialized LLM cannot emit a path outside the KG-Trie.

Decomposition, verification, and constrained decoding. Chain-of-thought, self-consistency, ReAct, and StructGPT motivate explicit reasoning steps and tool-based verification [18, 17, 20, 5]. Our DVI controller follows this spirit but moves the most fragile operation, candidate intersection, out of the LLM and into deterministic set operations over KG-derived candidates.

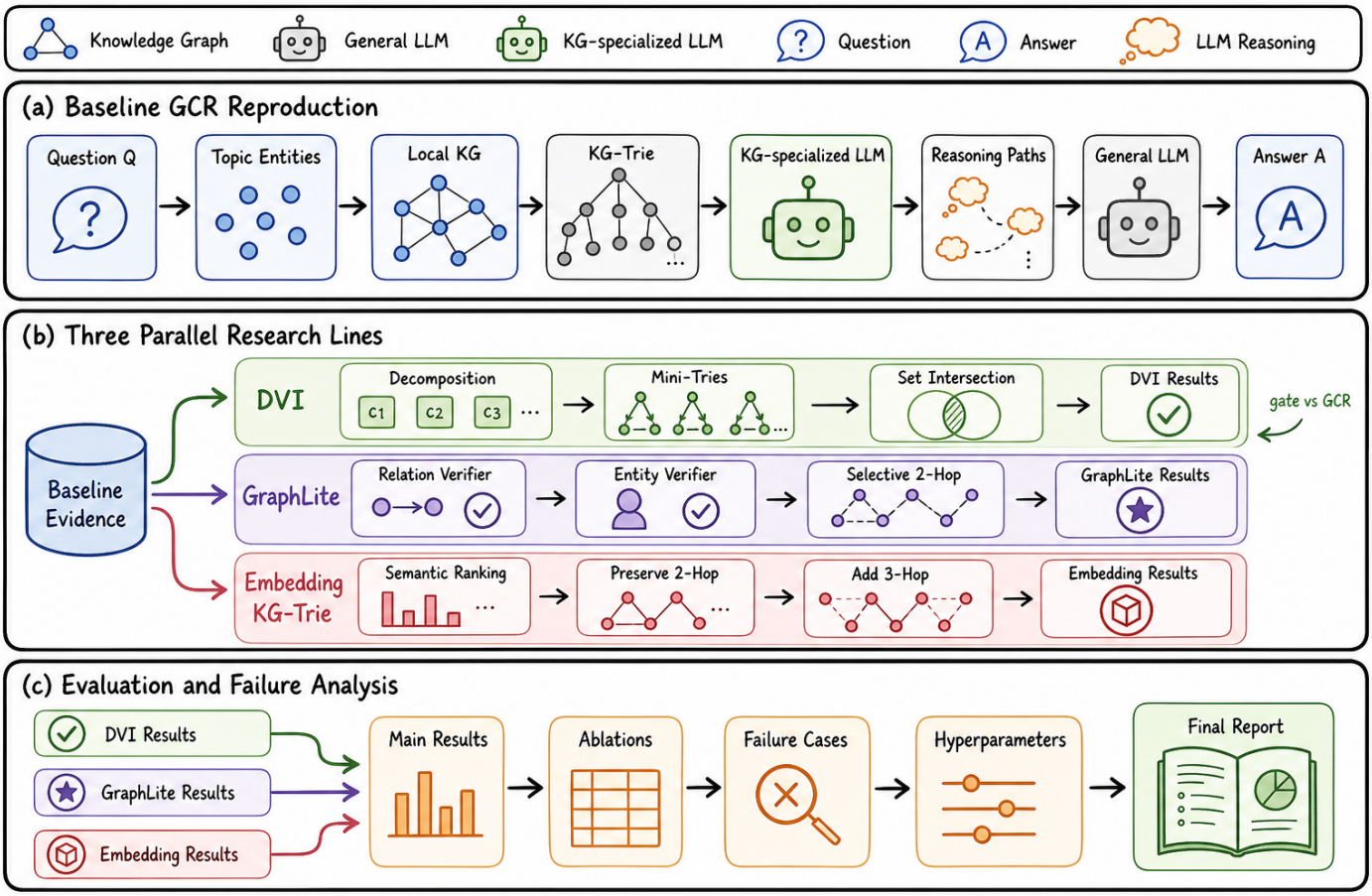

Figure 1: Project overview in a hand-drawn research-figure style. The three modification lines, DVI, GraphLite, and embedding-guided KG-Trie, are parallel research directions evaluated against the reproduced GCR baseline; only DVI includes an internal gate against GCR.

4 Baseline and Problem Formulation

Given a question q, topic entities E_q, and a local KG subgraph G_q, the goal is to predict an answer set $\hat{A}$. GCR constructs paths P_q by DFS from topic entities up to hop limit L, tokenizes serialized paths, and builds a trie T_q. During generation, the KG-specialized LLM is constrained to produce continuations accepted by T_q.

The distinction central to this project is:

$$\text{path validity} : p \in P_q \quad \not\Rightarrow \quad \text{answer correctness.} \tag{1}$$

For a decomposed question with constraints $c_1, \ldots, c_m$, the desired answer should satisfy:

$$e \in \bigcap_{i=1}^{m} S_i, \qquad \tau(e) = \tau_{ans}, \qquad F(e) = \text{true}, \tag{2}$$

where S_i is the entity set satisfying constraint c_i, τ_{ans} is the expected answer type, and F represents pending temporal, numeric, or superlative filters. A prefix trie tracks graph-valid token continuations, but it does not explicitly store $\{S_i\}$, τ_{ans}, or F.

Why prefix constraints are not enough. A trie constraint is a syntactic and graph-structural object. It knows that after a serialized entity token sequence the model may continue with one of the outgoing relation strings that appears in the local path set. It does not know whether this relation answers the user's predicate, whether a path tail has the requested answer type, or whether two paths from different topic entities refer to the same answer. As a result, constrained decoding can produce faithful evidence for the wrong subproblem.

Candidate-set view. For complex questions, the useful abstraction is a candidate set per constraint. If a question asks for countries that contain a department and are located in Central America, one constraint produces countries connected to the department and another produces countries in Central America. The desired operation is set intersection over entity identifiers or normalized entity names. If the system instead asks a final LLM to infer this intersection from a list of paths, the LLM may choose an entity that appears in only one branch or follow a generic path from a type node.

Operator view. Some CWQ questions require operators rather than only intersections. Comparative and superlative questions may ask for the earliest, latest, largest, or smallest item. Temporal questions may require checking whether a person held an office during a specific year. These operations should eventually be compiled into deterministic functions over KG attributes. The current project records such filters in the DVI decomposition but does not fully execute them, which is why operator failure remains a major limitation.

Baseline reproduction protocol. The reproduced GCR baseline keeps the original two-stage structure: first generate graph-constrained reasoning paths with a KG-specialized model, then ask a general LLM to produce final answers from those paths. The full-test runs use RoG-CWQ, hop length 2, zero-shot group beam decoding, $k = 10$, and GPT-4o mini for final answer aggregation. This setup makes the method comparisons meaningful because DVI, GraphLite, and embedding-guided KG-Trie are evaluated against the same basic GCR interface.

5 Methods

5.1 Method 1: DVI

DVI stands for Decompose-Verify-Intersect. It wraps the GCR verifier with a controller that tracks constraint state.

1. **Decompose**. A general LLM converts the question and topic entities into a JSON list of atomic relation, attribute, and filter constraints. If decomposition fails or the question has a single topic entity, DVI falls back to a GCR-like single global constraint.
2. **Verify**. For each relation-like constraint, DVI builds a mini-trie from DFS paths starting at the constraint anchor. The KG-specialized LLM decodes paths under this mini-trie. In fallback mode, DVI uses the original global trie.
3. **Intersect**. Candidate tail entities are extracted from each constraint's generated paths. DVI computes a strict intersection in Python. If empty, it records relaxed-intersection diagnostics instead of asking the LLM to guess the intersection.
4. **Answer**. A final general LLM receives evidence paths plus candidate entities and returns answer strings. An optional answer-aware module expands mediator endpoints using local KG edges when the question asks for a typed target such as a currency, language, school, team, or person.

The implemented state can be summarized as:

$$s_t = (e_t, p_t, \mathcal{C}_{sat}, \tau_{ans}, F), \tag{3}$$

where e_t is the current entity, p_t is a path prefix, $\mathcal{C}_{sat}$ is the set of satisfied constraints, τ_{ans} is the target answer type, and F stores filters. The current implementation instantiates this state at controller level rather than rewriting the full token-level decoding kernel.

Raw DVI is brittle: decomposition errors, mediator endpoints, and empty intersections can hurt strong baseline examples. We therefore use a confidence gate:

$$R(q) = \begin{cases} \text{DVI}(q), & |C_{DVI}(q)| = 2, \\ \text{GCR}(q), & \text{otherwise.} \end{cases} \tag{4}$$

We also evaluate looser gates $|C| \leq 1$, $|C| \leq 2$, and $|C| \leq 3$.

Decomposition format. The decomposer returns JSON with an `answer_variable` and a list of constraints. Relation and attribute constraints include an anchor entity and a natural-language hint; filter constraints include a predicate such as a date or numeric condition. The code validates that relation-like constraints have anchors and that filters have predicates. If the LLM response is unparsable or missing required fields, DVI falls back to one relation constraint per topic entity. For single-entity questions, the implementation also skips API decomposition and directly uses the fallback, because DVI has little intersection advantage there.

Mini-trie verification. For each relation or attribute constraint, `DVIPromptBuilder` builds a mini-trie by running DFS from only the anchor entity. This differs from the baseline global trie, where paths from all topic entities are mixed. The prompt shape intentionally stays close to the original GCR prompt because the KG-specialized model was tuned on `Question` and `Topic entities` fields; early experiments with extra instruction fields made the model echo conditions instead of generating paths. For simple or failed-decomposition cases, selective fallback builds the original global trie and marks the metadata field `selection_strategy` accordingly.

Candidate extraction and relaxation. The intersector removes `<PATH>` tags and answer blocks, extracts the last entity token from each path, and forms one candidate set per constraint. Strict intersection is attempted first. If it is empty, constraints are relaxed by keeping smaller candidate sets first, because smaller sets are treated as stronger constraints. The final prediction file stores strict size, relaxation status, dropped constraints, and final candidate count. These fields are later used for routing and failure analysis.

Answer-aware refinement. Some KG-valid paths end at intermediate entities. The answer-aware module infers a coarse answer type from question patterns, such as *currency*, *language*, *religion*, *person*, *film*, *team*, or *educational institution*. It then expands candidates through local KG edges whose relation names or entity types match the inferred answer type. This module is intentionally conservative: it improves some mediator cases but does not replace a full schema-aware executor.

5.1.1 DVI Algorithm

Figure 2 gives the implemented DVI control flow. The important distinction from baseline GCR is that DVI does not ask the final LLM to discover the intersection implicitly. It materializes candidate sets and records the intersection statistics before final answer generation.

5.1.2 Decomposition Details

The decomposer uses a general LLM with deterministic temperature to produce JSON. The schema contains an `answer_variable` and a list of constraints. Relation and attribute constraints require an anchor from the topic entities, while filter constraints contain a predicate-like description. The implementation includes a retry loop, cache support through `decompose_cache_path`, validation of required fields, and a single-entity fallback that avoids unnecessary API calls.

The decomposition stage is intentionally conservative. A bad decomposition can split a simple question into artificial constraints and remove useful evidence. Therefore DVI uses selective fallback when there is only one relation constraint, when decomposition fails, or when the question has no real multi-constraint structure. This is one reason raw DVI and routed DVI differ: raw DVI applies the full pipeline broadly, while routed DVI only trusts DVI answers when the final candidate set has a stable exact-size signature.

5.1.3 Candidate Extraction and Intersection

The intersector parses serialized GCR-style paths of the form entity, relation, entity, relation, entity. It strips path tags and answer blocks, extracts the last entity-like segment, deduplicates candidates, and computes Python set intersections. This simple operation is the core research intervention: the final LLM is no longer responsible for remembering which entity appeared under every constraint.

The implementation also records relaxed-intersection diagnostics. When a strict intersection is empty, this does not automatically mean the question is unanswerable. It may mean one constraint was decomposed incorrectly, one path pool missed the answer, or a mediator node was extracted instead of the final entity. Relaxed candidates are therefore diagnostic evidence for routing and analysis, not a proof of correctness.

5.1.4 Answer-Aware Refinement

Some valid KG paths end at intermediate nodes. For example, a path may reach a country when the question asks for its currency, or an office record when the question asks for the office holder. The answer-aware module uses lightweight question-type heuristics and local graph expansion to move from likely mediator endpoints toward typed answers. This is not a complete semantic parser; it is a practical patch for a common failure mode observed in CWQ. The report treats this as an incomplete but useful refinement layer.

5.2 Method 2: GraphLite

GraphLite is the efficiency-oriented verifier line. We use the name for two related code paths in the archive. The original `lite_framework.rar` prototype is an entity-level decoder; the integrated DVI experiment uses a lighter `PathScorer`. Both replace token-level KG-LLM constrained generation with explicit graph enumeration and neural scoring, but the entity-level prototype is more structured.

Entity-level Lite prototype. The core file `entity_level_decoder.py` first collects unique outgoing relations from the question topic entities and normalizes relation names into natural language. A trained `CrossEncoderVerifier` scores (q, r) pairs when available; otherwise the code falls back to LLM hidden-state similarity. After keeping the top relations, it enumerates one-hop candidate entities and scores (q, path) strings with an entity verifier. The decoder then performs selective two-hop expansion from all promising entities, not only Freebase MID nodes: with an entity verifier it expands MIDs or entities scoring above 0.3, and without one it uses a loose Llama log-probability threshold. This detail matters because the Lite report notes that many CWQ answers require a two-hop route through a non-MID intermediate entity.

Verifier training. Both Lite verifiers use the same BERT/DeBERTa-style cross-encoder class with a binary classification head and BCE loss [2, 4]. `train_cross_encoder.py` builds positives from gold KG paths and negatives from DFS paths that do not end at gold answers, with question-level train/validation splitting and hop-length-matched negatives. `train_entity_verifier.py` builds harder entity negatives from paths sharing the same first relation but ending at the wrong entity. At inference time, `entity_level_predict.py` can load both verifiers and skip the Llama model entirely, so Lite can run as a verifier-only graph enumerator.

Integrated PathScorer variant. The full-test result in Table 2 comes from the integrated PathScorer path rather than directly from the rar prototype. It enumerates DFS paths inside DVI, uses `all-MiniLM-L6-v2` as a bi-encoder to keep `bi_k=100` paths, and reranks them with `ms-marco-MiniLM-L-6-v2` to keep `cross_k=10`, following the sentence-embedding and MiniLM model family used in modern retrieval pipelines [10, 16]. The query representation concatenates the question and the constraint hint when available. This variant is easier to plug into DVI metadata and routing, while the rar prototype explains the fuller GraphLite design: relation selection, entity scoring, and two-hop expansion as explicit decisions.

5.2.1 GraphLite Algorithmic Motivation

GraphLite changes the unit of search. Baseline GCR searches in token space under a trie. GraphLite searches

Algorithm 1: DVI inference for one question.

1. Input question q, topic entities E_q, local graph G_q, KG model M_{KG}, general model M_G.
2. Decompose q into a JSON object with answer variable, relation/attribute constraints, and filter constraints. If decomposition is unavailable or not useful, create a fallback global constraint.
3. For each relation-like constraint c_i, collect DFS paths from the relevant anchor entity. Either build a mini-trie and decode with M_{KG}, or rank enumerated paths with PathScorer.
4. Extract tail entities from accepted paths to form S_i. Record path counts, trie statistics, and whether selective fallback was used.
5. Compute $C_{strict} = \bigcap_i S_i$. If C_{strict} is empty and `min_candidates` is positive, compute relaxed candidates by dropping constraints in a controlled diagnostic order.
6. Optionally apply answer-aware refinement to expand mediator endpoints toward typed final entities.
7. Build the final answer prompt from evidence paths plus candidate/intermediate entities. Generate concise answer strings with M_G.
8. Save answer, paths, final candidates, raw candidates, decomposition metadata, intersection metadata, and timing metadata.

Figure 2: DVI controller algorithm. The controller exposes intermediate state that is hidden in the baseline GCR path-to-answer pipeline.

in graph space by enumerating relations and entities, then scores structured candidates. This makes the verifier more inspectable: relation selection errors, entity scoring errors, and expansion errors can be analyzed separately.

The Lite prototype has three stages. Stage 1 enumerates first-hop relations from topic entities and scores question-relation pairs with a DeBERTa-style cross-encoder. Stage 2 enumerates entities reachable through selected relations and scores question-path-entity triples with an entity verifier. Stage 3 returns the highest-scoring entities and their supporting paths. The prototype also studies two-hop expansion, because many Freebase-style paths pass through mediator or MID nodes before reaching human-readable answers.

5.2.2 Integrated PathScorer Variant

The integrated code path is more lightweight than the full Lite prototype. Instead of training a complete entity-level decoder, `PathScorer` first uses `sentence-transformers/all-MiniLM-L6-v2` as a bi-encoder to filter a large path pool to `bi_k` candidates, then uses `cross-encoder/ms-marco-MiniLM-L-6-v2` to rerank to `cross_k`. The query is built from the full question plus the constraint hint when available. This design trades completeness for speed: it avoids KG-LLM decoding, but it depends on path text matching the question semantics.

5.2.3 Why GraphLite Underperforms End-to-End

GraphLite's lower F1 is not simply a failed implementation. It reveals a real modeling issue. A cross-encoder can identify paths that are semantically similar to the question, but KGQA often requires factual knowledge, answer typing, and multi-constraint aggregation. A path that mentions a country, anthem, or office may be semantically related but still answer the wrong predicate. This is why the path-reranker training improves Hit@1 and MRR at the path level while the final QA score remains below strong GCR baselines. The best future role for GraphLite is therefore a verifier, prefilter, or feature generator inside a larger DVI-style controller.

5.3 Method 3: Embedding-Guided KG-Trie

The embedding method modifies KG-Trie construction before decoding. The baseline collects all paths up to a hop limit, which can expose the decoder to many generic or irrelevant paths. Our embedding-guided variant ranks paths by semantic similarity between the question and a textual representation of each path, then keeps only a budgeted subset for trie construction.

The enhanced code also implements a hybrid reranker combining embedding similarity, relation keyword overlap, entity keyword overlap, shared-tail coverage, source specificity, and a small length penalty. Generic-source filtering removes paths starting from type-like entities such as *College/University* when more specific topic entities are available. Finally, `output_topk` keeps only the top generated candidates before evaluation, turning a path-generation system into a more precision-oriented answer system.

The most useful version is coverage-preserving 3-hop expansion: protect strong 2-hop coverage and use 3-hop embedding-ranked paths to add missing evidence. This matters because naive 3-hop expansion creates path explosion and lowers precision.

Path serialization for embeddings. Each KG path is converted into text by concatenating the head entity, relation tokens, and tail entities, with relation names split on dots and underscores. The question and path texts are encoded by `sentence-transformers/all-MiniLM-L6-v2` [10, 16]. Paths are ranked by cosine similarity to the question embedding, and only the top budgeted paths enter the trie. This keeps trie construction compatible with the original GCR decoder: after ranking, decoding is still graph-constrained by a Marisa trie.

Hybrid scoring features. The enhanced implementation adds lexical and structural features to the embedding score. Relation overlap measures whether relation-name tokens match question words; entity overlap measures whether path entities match question words; shared-tail coverage rewards terminal entities reachable from multiple topic entities; source specificity downweights generic type-like sources; and a small length penalty discourages long noisy paths. In the archived CWQ20 split, these features are useful as diagnostics but do not clearly beat pure embedding ranking, so the final report treats them as an ablation rather than the main method.

5.3.1 Embedding-Guided Path Selection Algorithm

Embedding-guided KG-Trie changes path admission, not final answer aggregation. The baseline constructs a trie from DFS paths around topic entities. The modified builder serializes each candidate path into readable text, embeds the question and path strings, computes semantic similarity, and uses this score as part of the path ranking. Hybrid variants add relation overlap, entity overlap, path frequency, source-generic penalties, and path length controls.

The key engineering separation is between *collection budget* and *trie budget*. A larger collection budget allows the system to look beyond short local paths, especially at 3 hops. A smaller trie budget prevents the decoder from being overwhelmed by irrelevant branches. This separation is what makes preserve-2 plus emb-3 meaningful: it keeps the reliable 2-hop evidence while allowing selected 3-hop evidence to enter.

5.3.2 Why Naive 3-Hop Expansion Fails

A naive 3-hop trie often hurts because the number of paths grows rapidly and generic source nodes create many plausible distractors. The model then sees more valid continuations but less useful discrimination. The CWQ20 ablation makes this clear: budgeted 3-hop without embedding has lower F1 than the 2-hop beam baseline, while embedding-guided top-1 selection improves F1. The correct interpretation is not that longer paths always help; they help only when the system controls which longer paths are admitted and how final candidates are selected.

5.4 Worked Question-to-Answer Trace

Figure 4 compares the three method-specific traces on one compositional question: "The national anthem Afghan National Anthem is from the country which practices what religions?" The correct trace is not a single topical path about the anthem. It must bind the anthem to its country, keep the target answer type as *religion*, and then return the religions connected to that country: *Sunni Islam* and *Shia Islam*.

In DVI terms, the decomposer should produce two relation constraints, anthem → country and country → religion. Mini-tries verify each branch, the bridge country candidate is carried forward, and answer-aware refinement checks that final candidates are religions. In GraphLite terms, relation and entity verifiers should downweight the tempting but wrong language path and keep country/religion paths. In embedding-guided KG-Trie terms, semantic ranking should keep the country/religion evidence in the trie while preventing generic music-language paths from dominating the constrained decoder.

6 Implementation Details

The implementation keeps DVI, GraphLite, and embedding-guided KG-Trie as separate code paths so that failures can be attributed to the controller, verifier, or trie-construction stage. DVI prediction files store both answers and metadata: decomposition status, relation/filter constraints, per-constraint path counts, trie statistics, strict and relaxed intersection sizes, candidate sets, fallback mode, answer-type hints, and timing for decomposition, KG decoding, path scoring, intersection, and final answering. This metadata is what makes post-hoc routed DVI possible without rerunning the KG model.

Two engineering choices matter most. First, DVI uses selective fallback: if a question has only one relation constraint or decomposition fails on a single-topic example, the system keeps the original global GCR trie instead of forcing mini-tries that have no intersection advantage. Second, embedding-guided KG-Trie separates the number of DFS paths collected from the number admitted into the trie; on 3-hop CWQ probes, this budget is essential because raw candidate pools can exceed hundreds of thousands of paths. GraphLite changes the verifier side instead: it trades KG-LLM decoding for explicit graph enumeration plus relation/entity/path scoring, which explains the much lower runtime and the current quality drop.

6.1 Prediction Metadata and Saved State

A major implementation difference between the baseline and the modified systems is metadata. The DVI workflow writes not only final answers but also a structured record of how each answer was produced. Important fields include the decomposition object, decomposition metadata, relation constraints, filter constraints, path counts per constraint, trie statistics, raw final candidates, refined final candidates, intersection statistics, answer-aware refinement statistics, evidence paths, fallback mode, and timing.

This metadata serves three purposes. First, it makes failure analysis possible without rerunning expensive models. Second, it enables post-hoc routing experiments such as exact-size routed DVI and oracle routing. Third, it provides features for a future learned router. A robust router should not look only at candidate count; it should combine decomposition status, strict-intersection size, relaxation depth, path-score margins, answer-type confidence, baseline confidence, and timing.

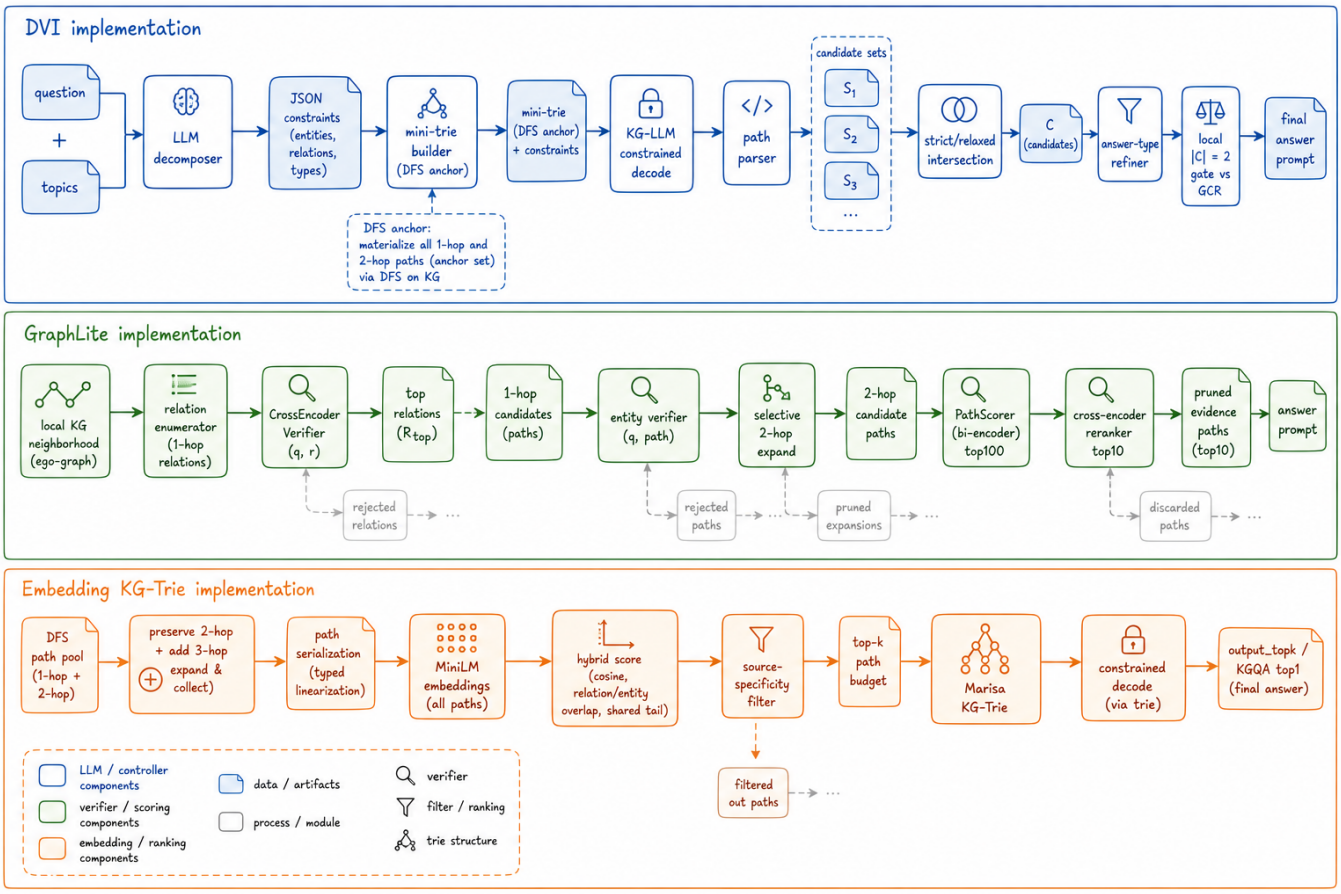

Figure 3: Implementation-level method architecture. Unlike the project overview, this figure expands each method into internal modules, intermediate data structures, and control decisions: DVI tracks constraints and candidate sets, GraphLite replaces KG-LLM decoding with relation/entity/path verifiers, and embedding-guided KG-Trie ranks and budgets paths before trie construction.

6.2 Code Organization

The GitHub artifact keeps the three method tracks separated. The main implementation is in `code/dvi_gcr`; the embedding-guided KG-Trie patch is in `code/embedding_guided_kgtrie`; and the standalone GraphLite prototype is in `code/graphlite`. The separation is deliberate: DVI, GraphLite, and embedding-guided path selection answer different engineering questions and should be testable independently before being merged into one pipeline.

6.3 Caching, Runtime, and Reproducibility

DVI uses a decomposition cache to avoid repeated API calls. Full-test runs can be expensive because they combine local KG model inference with API-backed decomposition and final answering. The timing summaries in the archive are therefore part of the evidence: they show which parts of the pipeline dominate runtime. GraphLite's advantage is most visible here, because explicit path scoring avoids the slowest KG-LLM constrained decoding step.

7 Experimental Setup

We evaluate mainly on RoG-CWQ. The full test split contains 3531 examples. We also use RoG-WebQSP to test whether embedding-guided path selection transfers to a simpler KGQA benchmark. Metrics are Precision, Recall, F1, Hit@1, and Hit. The archived evaluation script also reports Accuracy, which aligns with answer-set recall in these outputs. Timing is mean wall-clock seconds per example when prediction files contain timing metadata.

The reproduced full CWQ baselines use hop length 2, zero-shot group beam search, $k = 10$, and GPT-4o mini as the final answer LLM. We compare three KG-specialized LLM checkpoints: Llama-3.1-8B, Llama-2-7B, and Qwen2-0.5B [3, 15, 19]. Embedding-guided GCR uses Qwen2-0.5B for method-only comparison and Meta-Llama-3.1-8B for base-model exploration. DVI uses both Llama-3.1 and Llama-2, plus a Qwen ablation. GraphLite uses the PathScorer verifier without a KG-specialized generation model.

7.1 Datasets and Splits

The main benchmark is RoG-CWQ, derived from ComplexWebQuestions. It stresses composition, conjunction,

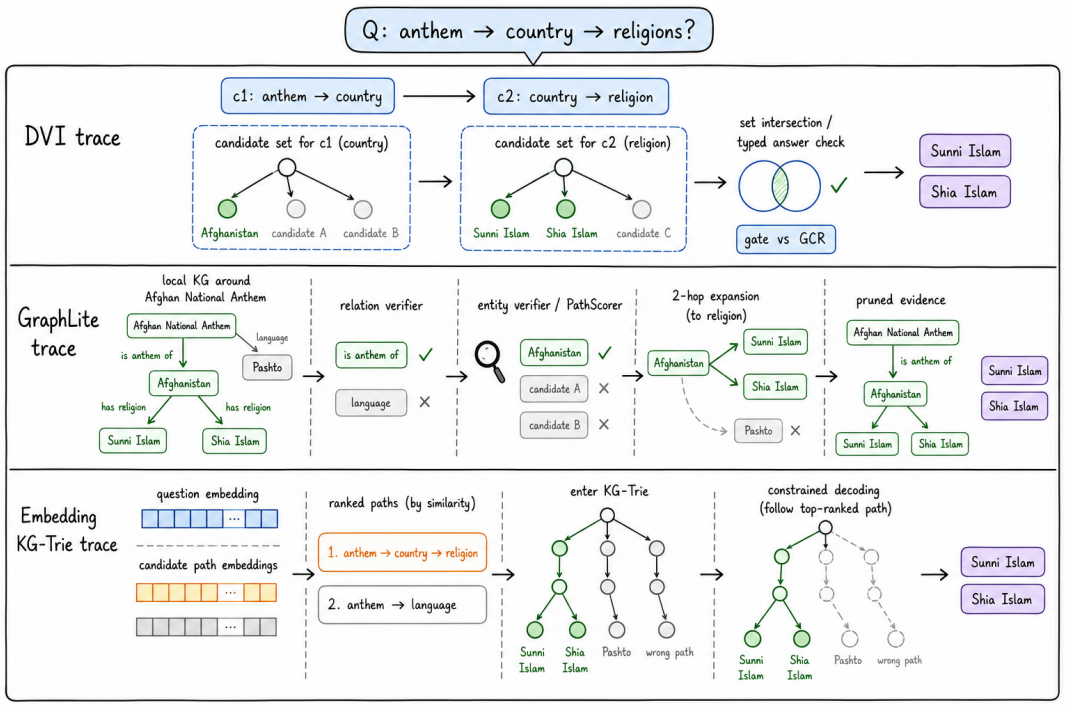

Figure 4: Three separate question-to-answer traces for the same anthem-country-religion query. DVI traces decomposition and candidate intersection, GraphLite traces relation/entity verification and pruning, and embedding-guided KG-Trie traces semantic path ranking before constrained decoding.

Metadata group	Purpose
Decomposition	Stores JSON constraints, answer variable, fallback reason, cache hit status, and API-call count.
Path retrieval	Stores per-constraint path lists, path counts, selection strategy, trie statistics, and whether the KG-LLM or PathScorer verifier was used.
Candidate sets	Stores raw tail-entity candidates, strict intersection size, relaxed candidates, and final refined candidates.
Answer generation	Stores evidence paths shown to the final LLM and parsed final answer lines.
Timing	Stores total time plus decomposition, KG decoding, path scoring, intersection, refinement, and final-answer times.
Routing features	Stores candidate-set size, fallback mode, relation/filter counts, and other signals useful for hand-written or learned gates.

Table 1: DVI metadata groups. Saving this state is essential for interpreting why a prediction succeeds or fails.

comparative, and superlative reasoning over Freebase-style graph evidence. The full test split contains 3531 examples. WebQSP is used for selected embedding-guided experiments because it is generally simpler and has shorter reasoning patterns, which helps isolate whether path-selection changes transfer beyond CWQ.

7.2 Models

The baseline reproduction and DVI experiments use three KG-specialized checkpoints from the GCR setting: GCR-Meta-Llama-3.1-8B-Instruct, GCR-Llama-2-7b-chat-hf, and GCR-Qwen2-0.5B-Instruct. GPT-4o mini is used as the general final-answer model and as the judge in the archived evaluation protocol. GraphLite/PathScorer uses sentence-transformer retrieval and cross-encoder reranking rather than a KG-specialized generation model.

7.3 Metrics

The report uses Accuracy/Recall, Hit, Hit@1, F1, Precision, and mean wall-clock time per example. The archived evaluation script reports Accuracy in a way that aligns with recall-style answer coverage in these outputs. Hit measures whether any gold answer appears in the returned answer set. Hit@1 measures whether the top returned answer is correct. F1 and Precision capture answer-set quality, which matters when a system returns multiple entities.

7.4 Why Multiple Metrics Matter

A method can improve Hit while hurting Precision if it returns more candidates. It can improve Hit@1 while hurting F1 if it chooses one strong answer but omits additional gold answers. It can also improve F1 while being too slow for

practical use. This is why the report does not reduce the comparison to a single number. DVI is judged by full-test F1 and Hit@1, GraphLite by speed-quality tradeoff, and embedding-guided KG-Trie by whether additional hops improve controlled probes without sacrificing full-test behavior.

8 Main Results

Table 2 gives the full-test results. The three reproduced GCR baselines satisfy the project requirement to explore more than two KG-specialized LLMs. Llama-2 has the strongest Accuracy/Recall, Hit, and Hit@1; Llama-3.1 has the strongest baseline F1 and Precision; Qwen2-0.5B is much faster and weaker.

Raw DVI does not beat the strongest GCR baselines. It has useful examples, but the full DVI pipeline is vulnerable to decomposition mistakes and answer-incomplete mediator endpoints. Exact-size routed DVI is more stable. For Llama-3.1, it improves Accuracy/Recall from 59.01 to 59.26, Hit from 64.34 to 64.68, Hit@1 from 55.88 to 56.22, F1 from 54.55 to 54.63, and Precision from 54.87 to 54.90. For Llama-2, it improves Accuracy/Recall from 60.51 to 60.56, Hit from 66.07 to 66.27, F1 from 54.50 to 54.51, and Precision from 54.29 to 54.42, while lowering Hit@1 from 57.12 to 56.92. These gains are small, so the result should be interpreted as evidence for a useful routing direction rather than a decisive leaderboard improvement.

GraphLite is the fastest full-test verifier among our modified methods. It reduces mean inference time from 12.72s for Llama-3.1 raw DVI to 3.09s, but F1 drops from 52.57 to 42.14. This makes GraphLite useful as an efficiency ablation and possible prefilter, but not yet a replacement for KG-LLM verification.

8.1 Detailed Interpretation of the Full-Test Table

The full-test table has three layers. The first layer is baseline reproduction. Llama-2 and Llama-3.1 are close in F1, with Llama-2 slightly stronger on Hit and Hit@1 and Llama-3.1 slightly stronger on Precision. Qwen2-0.5B is much faster and substantially weaker, which confirms that model capacity is a major factor.

The second layer is raw DVI. Raw DVI has lower F1 than the corresponding GCR baselines for both Llama-3.1 and Llama-2. This means that decomposition plus intersection is not automatically better than a strong global-trie baseline. The raw pipeline can lose useful global paths, extract mediator endpoints, or trust an incomplete decomposition.

The third layer is routed DVI. The exact-size gate improves Llama-3.1 F1 from 54.55 to 54.63 and Llama-2 F1 from 54.50 to 54.51. These are small gains, but they are important because they come from a simple rule that does not use gold labels. The oracle table then shows why this rule is conservative: DVI and GCR have enough complementary examples to reach above 61 F1 under a post-hoc oracle.

8.2 Statistical Caution

The aggregate routed DVI gain is small, so it should not be overstated. The evidence is better interpreted as a systems result: DVI creates useful intermediate state and complementary answers, but the current hand-written router is too weak to capture all of that signal. A future learned router should be evaluated with held-out validation and confidence intervals.

8.3 Oracle Routing Headroom

The strongest evidence for DVI is complementarity. On the 3520 examples shared by the Llama-3.1 baseline and raw DVI, a post-hoc oracle that chooses the prediction source with higher F1 reaches 61.51 F1, compared with 54.63 for the baseline on the same examples and 52.57 for raw DVI. DVI uniquely fixes 282 Hit@1 examples that GCR misses, while GCR uniquely fixes 348 that DVI misses. Llama-2 shows the same pattern: oracle F1 reaches 61.37, with 259 DVI-only Hit@1 fixes and 379 baseline-only fixes. This means DVI contains useful signal, but a better learned router is needed.

KG model	Source	N	Hit@1	F1	Hit
Llama-3.1	GCR	3520	56.05	54.63	64.43
Llama-3.1	Raw DVI	3520	54.18	52.57	64.32
Llama-3.1	Oracle	3520	**62.36**	**61.51**	**71.45**
Llama-2	GCR	3520	57.27	54.57	66.16
Llama-2	Raw DVI	3520	53.86	52.31	64.35
Llama-2	Oracle	3520	**63.35**	**61.37**	**72.39**

Table 3: Post-hoc oracle routing over shared GCR/DVI examples. This is an upper bound, not a deployable method.

9 Embedding-Guided KG-Trie Results

Table 4 summarizes the embedding line. The first full-test attempt simply inserted semantic ranking into 2-hop KG-Trie construction. This nearly ties the Qwen baseline but does not improve it: CWQ F1 is 23.76 vs. 23.86, and WebQSP F1 is 36.87 vs. 37.17. This negative result is informative: semantic similarity should not replace strong short-hop graph coverage.

The improved setting keeps 2-hop coverage and uses embedding ranking to add 3-hop evidence. On CWQ100, full candidates improve slightly in recall-style metrics, but the real gain appears after candidate fusion and KGQA-aware top-1 selection, reaching 39.68 F1. On WebQSP100, preserve-2 plus 3-hop embedding improves no-topic top-1 selection from 41.60 F1 for the baseline probe to 44.70 F1. On the controlled CWQ20 experiment, 3-hop embedding-guided top-1 reaches 45.56 F1, compared with 30.50 F1 for the strongest 2-hop beam baseline and 22.37 F1 for naive 3-hop without embedding. The pattern is consistent: extra hops help only when path explosion is controlled.

Method	KG/verifier	N	Acc./Rec.	Hit	Hit@1	F1	Prec.	Time(s)	Coverage
GCR baseline	Llama-3.1-8B	3531	59.01	64.34	55.88	54.55	54.87	2.06	882/3531
GCR baseline	Llama-2-7B	3531	60.51	66.07	**57.12**	54.50	54.29	6.27	3531/3531
GCR baseline	Qwen2-0.5B	3531	47.81	54.46	44.83	42.31	42.52	1.68	3531/3531
Raw DVI	Llama-3.1-8B	3520	57.71	64.32	54.18	52.57	53.42	12.72	3520/3520
Routed DVI, $\|C\| = 2$	Llama-3.1-8B	3531	59.26	64.68	56.22	**54.63**	**54.90**	6.58	1307/3531
Raw DVI	Llama-2-7B	3520	57.76	64.35	53.86	52.31	53.02	7.92	3520/3520
Routed DVI, $\|C\| = 2$	Llama-2-7B	3531	**60.56**	**66.27**	56.92	54.51	54.42	6.53	3531/3531
GraphLite / PathScorer	Entity / path verifier	3520	51.05	56.19	45.09	42.14	40.64	**3.09**	3520/3520

Table 2: Full-test RoG-CWQ results. Values are percentages except time. Routed DVI uses DVI only when the final candidate set has exactly two entities; otherwise it keeps the corresponding GCR prediction.

Dataset	Method	Setting	Acc./Rec.	Hit	F1	Precision
CWQ full	Qwen GCR	2-hop baseline	55.60	62.50	**23.86**	17.73
CWQ full	Qwen embedding	2-hop baseline-aligned	55.48	62.39	23.76	17.66
WebQSP full	Qwen GCR	2-hop baseline	68.39	87.71	**37.17**	33.58
WebQSP full	Qwen embedding	2-hop baseline-aligned	67.75	87.77	36.87	33.44
CWQ100	preserve-2 + emb-3	all candidates	63.42	70.00	27.48	20.27
CWQ100	baseline + emb-3 fusion	KGQA top-1	39.13	47.00	**39.68**	**45.00**
WebQSP100	preserve-2 + emb-3	all candidates	70.89	90.00	34.18	31.02
WebQSP100	preserve-2 + emb-3	no-topic top-1	42.76	66.00	**44.70**	**64.00**
CWQ20	3-hop embedding	output top-1	43.79	55.00	**45.56**	**55.00**

Table 4: Embedding-guided KG-Trie results. Pure 2-hop embedding selection is close to, but does not beat, the full Qwen baseline. It becomes useful when 2-hop coverage is preserved and 3-hop evidence is added with candidate reranking.

9.1 Embedding Experiments as Negative and Positive Evidence

The embedding-guided results contain both negative and positive evidence. The negative result is the full-test 2-hop setting: semantic path selection is close to the Qwen baseline but does not beat it. This shows that semantic similarity is not a replacement for graph coverage. If a short, exact KG path is removed because its surface text is not semantically obvious, the downstream model may lose the answer.

The positive result appears when embedding is used to add longer evidence rather than replace short evidence. Preserve-2 plus emb-3 follows this principle. The system keeps the original short-hop evidence and admits selected 3-hop paths only when their semantic score or hybrid score justifies the extra budget. The CWQ20 and WebQSP100 probes show that this can improve top-1 candidate quality.

9.2 Implications for a Unified System

In a future unified system, embedding-guided KG-Trie should run before GraphLite and DVI. It should build a broader evidence pool with metadata such as source entity, hop length, semantic score, relation overlap, and generic-source flags. GraphLite can then score and prune this pool. DVI can consume the resulting paths as constraint-specific evidence and perform deterministic candidate intersection. This ordering preserves the strengths of all three methods.

10 Failure Analysis

10.1 Failure Analysis Methodology

The failure analysis combines automatic grouping with manual inspection. The automatic part uses compositionality labels and routed-DVI deltas to identify which question types improve or degrade. The manual part inspects concrete predicted paths and compares them with required reasoning steps. This is necessary because a KG-valid path can look superficially reasonable while answering the wrong predicate.

The analysis focuses on where the pipeline first diverges from the required reasoning chain. If the correct entity never appears in the path pool, the failure is retrieval. If it appears but is ranked below distractors, the failure is selection. If evidence for multiple constraints exists but the final answer does not combine them, the failure is aggregation. If the output is related but has the wrong type, the failure is typing. If the answer requires a date, numeric comparison, or superlative operation that is not executed, the failure is operator execution.

10.2 By Compositionality Type

Table 5 reports changes in percentage points for exact-size routed DVI against its own GCR baseline. Routed DVI helps Llama-3.1 on composition and comparative Hit@1, while superlative questions remain difficult. Llama-2 shows

small gains on composition, comparative F1, and superlative F1, but loses on conjunction. This mixed picture explains why the aggregate gain is small: decomposition and intersection help some complex patterns but can also drop correct evidence when the constraint representation is incomplete.

Type	N	Llama-3.1 Δ		Llama-2 Δ	
		Hit@1	F1	Hit@1	F1
composition	1546	+0.84	+0.55	+0.19	+0.26
conjunction	1575	-0.19	-0.12	-0.76	-0.38
comparative	213	+1.88	-0.01	+0.94	+0.70
superlative	197	-1.02	-1.92	+0.00	+0.54

Table 5: Compositionality-type change for exact-size routed DVI versus the corresponding GCR baseline.

10.3 Detailed Failure Inference Steps

The rubric asks for detailed inference steps on multi-constraint failures. Table 6 gives representative examples from the archived predictions and shows exactly which step breaks. The common pattern is that the generated paths are KG-valid, but the system either starts from a generic topic entity, follows the wrong relation, or fails to execute an intersection or filter.

The first and fourth rows motivate embedding-guided path selection: generic sources such as *Country* or *College/University* create many valid distractors, so semantic ranking and generic-source filtering are useful. The second row motivates relation-aware scoring: topic similarity alone is insufficient because the anthem-language path is semantically close but predicate-wrong. The third row motivates DVI: multiple topic entities must be combined through answer-level intersection rather than left to the final LLM. The fifth row is the main remaining gap: neither DVI nor GraphLite fully executes temporal and numeric filters yet.

10.4 Error Taxonomy Across Methods

The failure cases can be organized by which part of the pipeline failed.

Retrieval failure. The needed entity or relation path is absent from the local path pool. This happens when the hop limit is too small, when the relevant bridge is beyond the DFS budget, or when topic entity linking gives a generic type node. Embedding-guided 3-hop expansion primarily targets this failure mode.

Selection failure. The correct path exists in the pool but is ranked below distractors. GraphLite and embedding-guided ranking both target this failure mode. The path-reranker training results show that supervised scoring can substantially improve path-level Hit@1, but QA still requires downstream aggregation.

Aggregation failure. The evidence contains paths for several constraints, but the system fails to combine them into a single answer. This is the main motivation for DVI. GCR leaves most aggregation to the final LLM; DVI turns at least the entity-intersection part into deterministic Python set operations.

Typing failure. The system finds a related entity but not an entity of the requested answer type. Examples include returning a language when the question asks for a religion, a country when it asks for a currency, or a mediator record when it asks for a person. Answer-aware refinement targets this failure mode, but the current rules are incomplete.

Operator failure. The system retrieves candidates that satisfy relation constraints but does not execute date, numeric, comparative, or superlative operators. This failure mode remains mostly unsolved in the current implementation. It requires compiling filters into executable comparisons over KG attributes.

This taxonomy explains why the methods are complementary. Embedding-guided KG-Trie helps retrieval and selection, GraphLite helps selection and speed, and DVI helps aggregation. None of them fully solves typing and operator execution yet.

11 Ablation and Hyperparameter Analysis

The project rubric requires ablations beyond the main-result table. We therefore keep the main ablation tables in the body and use them to diagnose calibration, verifier quality, and path-budget sensitivity.

11.1 DVI Gate Thresholds

Table 7 compares hand-written DVI confidence gates. Singleton DVI outputs often look precise but can be mediator endpoints, while broad gates reintroduce noisy alternatives. Exact-size routing, $|C| = 2$, is the most stable deployed rule across the two stronger KG-specialized LLMs.

Model	Gate	Acc./Rec.	Hit@1	F1	Time(s)
Llama-3.1	$\|C\| \leq 1$	59.03	54.80	54.33	8.54
Llama-3.1	$\|C\| = 2$	59.26	56.22	**54.63**	6.58
Llama-3.1	$\|C\| \leq 2$	**59.28**	55.14	54.41	10.29
Llama-3.1	$\|C\| \leq 3$	58.92	54.63	53.87	11.27
Llama-2	$\|C\| \leq 1$	60.40	55.88	54.38	6.89
Llama-2	$\|C\| = 2$	**60.56**	**56.92**	**54.51**	6.53
Llama-2	$\|C\| \leq 2$	60.45	55.68	54.40	7.16
Llama-2	$\|C\| \leq 3$	59.71	54.91	53.84	7.34

Table 7: DVI confidence-gate ablation. Exact-size routing is the most stable threshold across the two stronger KG-specialized LLMs.

11.2 GraphLite Verifier Training

GraphLite tests whether expensive KG-LLM verification can be replaced by explicit path enumeration and neural

Case	Required inference steps	Observed KG-valid path / output	Failure point
Country Nation World Tour college	1. Find the artist of *Country Nation World Tour*. 2. Follow the artist's education relation. 3. Return the institution, gold *Belmont University*.	The model follows a generic type source: *College/University* → *College* → *films* → *Johnny Be Good* → *country* → *United States*.	Generic-entity retrieval and answer-type failure. The path is valid but never identifies the tour artist.
Afghan National Anthem religions	1. Map the anthem to its country. 2. From that country, retrieve practiced religions. 3. Return *Sunni Islam* and *Shia Islam*.	The model follows a topical but wrong relation: *Afghan National Anthem* → *language* → *Pashto language*. Some paths continue to Afghanistan but return official language.	Relation-selection failure. The path is about language rather than religion.
Libya anthem leader	1. Identify the country whose anthem is *Libya, Libya, Libya*. 2. Intersect with the office/person constraint from *Prime Minister of Libya*. 3. Return the leader, gold *Abdullah al-Thani*.	Predicted paths include *Prime Minister of Libya* → *office holders* → *position record* → *jurisdiction* → *Libya* → *languages spoken* → *Arabic*.	Aggregation and typing failure. The system reaches Libya but drifts to language, not the office holder.
Alta Verapaz and Central America	1. Candidate country must contain *Alta Verapaz Department*. 2. Candidate must also be in *Central America*. 3. Return the intersection, gold *Guatemala*.	Baseline follows paths from generic *Country*, e.g., fictional country settings ending in *Europe*.	Multi-entity intersection failure. Generic paths overwhelm the specific department/region evidence.
Temporal governor filter	1. Retrieve governors of Arizona. 2. Check who held office in 2009. 3. Apply the predicate "held governmental position before 1998".	Graph paths can retrieve governor/office-holder candidates, but the date predicates are not executed by the trie.	Operator failure. Prefix validity does not evaluate temporal constraints.

Table 6: Detailed inference-step failures. Each error is KG-valid at the path level but invalid at the full-question constraint level.

scoring. Fine-tuning the cross-encoder improves path-level ranking, but endpoint supervision still does not guarantee full answer satisfaction.

Verifier	Groups	MRR	Hit@1	Hit@5	Hit@10
Pretrained cross-encoder	158	68.04	56.33	85.44	88.61
Fine-tuned cross-encoder	158	**77.13**	**69.62**	**87.97**	**91.77**

Table 8: GraphLite path-reranker training metrics on held-out grouped path candidates. Values are percentages except group count.

11.3 Hop Length and Candidate Selection

Embedding-guided KG-Trie has the opposite calibration problem: extra hops improve coverage but create many distractors. Table 9 shows that naive 3-hop expansion hurts, while embedding-guided selection makes 3-hop evidence useful.

CWQ20 setting	Hit	F1	Prec.	Recall
2-hop group-beam baseline	45.00	27.83	30.00	36.88
2-hop beam early stopping	45.00	30.50	34.17	36.88
3-hop budgeted, no embedding	45.00	22.37	20.83	31.29
3-hop embedding, all candidates	**60.00**	37.00	37.50	**51.88**
3-hop embedding, top-1	55.00	**45.56**	**55.00**	43.79

Table 9: Hop-length and output-selection ablation on CWQ20. Extra hops help only when semantic selection and candidate filtering control path explosion.

Across the ablations, the recurring pattern is calibration: DVI needs calibrated routing, GraphLite needs calibrated verifier scores, and embedding-guided KG-Trie needs calibrated path budgets.

11.4 What the Ablations Show Together

The three ablation groups diagnose different calibration problems. DVI gate ablations diagnose when to trust the controller. GraphLite verifier training diagnoses whether learned path scoring improves path-level ranking. Hop-length ablations diagnose how much evidence can be added before path explosion harms answer quality.

The common lesson is that the methods are not plug-and-play improvements. DVI needs a calibrated router; GraphLite needs calibrated verifier scores and probably an answer-type layer; embedding-guided KG-Trie needs calibrated path budgets. A future system should expose these calibration variables as first-class features rather than burying them inside a single prediction file.

12 Discussion

The results support four conclusions.

First, GCR is a strong baseline and the KG-specialized model matters. Llama-2 and Llama-3.1 are far stronger than Qwen2-0.5B on final-answer CWQ metrics. This satisfies the base-LLM exploration requirement and also shows that method comparisons must control for model capacity.

Second, DVI should be selective. Raw DVI has useful answers but is not robust enough to replace GCR globally. Exact-size gating gives small full-test gains, while oracle routing shows large headroom. The next version should train a router from DVI metadata, question type, candidate count, strict-intersection size, relaxation status, answer type, and baseline confidence.

Third, GraphLite exposes the speed-quality frontier. Explicit relation/entity enumeration plus neural scoring is much faster than KG-LLM DVI, but loses quality. Fine-tuned path reranking improves grouped verifier Hit@1 from 56.33% to 69.62% on held-out path groups, yet QA performance remains below the strongest GCR/DVI methods. This suggests GraphLite is best used as a prefilter or auxiliary verifier, not a standalone answer system.

Fourth, embedding-guided KG-Trie is a useful path-construction module only when coverage is protected. Pure 2-hop semantic pruning is too aggressive; preserve-2 plus 3-hop embedding expansion is more promising. The method is complementary to DVI because it improves the evidence pool, while DVI improves constraint aggregation.

13 Detailed Design Lessons

Lesson 1: Faithfulness and correctness are different. GCR improves faithfulness by preventing unsupported path generation, but a faithful path can still be irrelevant. The report's failure cases repeatedly show KG-valid evidence that answers a nearby predicate rather than the requested predicate. Therefore the next step for faithful KGQA is not only stricter graph grounding but richer state about constraints and operators.

Lesson 2: Intersections should be programmatic. Multi-entity conjunctions are fragile when left to a final LLM. The final model may attend to the most fluent path or the most familiar entity rather than computing an actual set intersection. DVI's main contribution is to make this operation explicit, inspectable, and reproducible.

Lesson 3: Longer paths need admission control. Increasing hop length is tempting because it improves coverage, but the path space expands quickly and generic entities dominate. Embedding-guided selection is useful when it acts as admission control for additional evidence, not when it replaces all short-hop graph coverage.

Lesson 4: Fast verifiers need answer semantics. GraphLite demonstrates that relation/entity enumeration plus cross-encoder scoring can be much faster than KG-LLM decoding. However, path relevance is not the same as answer correctness. A fast verifier must eventually model answer type, multi-constraint satisfaction, and operator execution.

Lesson 5: Routing is a modeling problem. The oracle routing table shows large headroom, but exact-size routing captures only a small part of it. A learned router should be trained on validation data and should use features from all modules: baseline answer confidence, DVI candidate-set size, decomposition validity, path-score margins, answer-type confidence, and timing.

14 Limitations and Future Work

The current system is a working prototype rather than a finished KGQA solver. Its main limitations are answer typing, executable filters, and routing calibration. Answer-aware refinement uses lightweight type heuristics and local expansion, but a stronger system should learn answer types and mediator schemas. Temporal, numeric, comparative, and superlative questions require executable operators over KG attributes; the current DVI decomposer records these filters but mostly treats them as metadata. Routing is also still simple: exact-size gating gives a small full-test gain, while the oracle shows much larger headroom. A learned router should use decomposition status, strict-intersection size, relaxation status, fallback mode, answer-type confidence, path-score margins, and baseline confidence.

The three methods should eventually be merged into one configurable pipeline. Embedding-guided KG-Trie can expand evidence beyond 2 hops, GraphLite can cheaply score and prune paths, and DVI can perform deterministic intersection and answer-type validation. A stronger evaluation should report confidence intervals, evaluate CWQ and WebQSP under the same final-answer protocol, and isolate gains on examples requiring multi-entity intersection, date comparison, or superlative choice.

Integration plan. The next version should combine the three methods without adding a project-level router. A coverage-preserving evidence pool would keep the original 2-hop GCR neighborhood, add only embedding-ranked 3-hop paths, and store metadata such as source entity, hop length, relation overlap, embedding margin, and generic-source flags. GraphLite can then score this pool before KG-LLM decoding, producing compact accepted and rejected evidence paths. DVI can consume those paths as constraint-specific candidate sets, run strict or relaxed intersections, and expose routing features such as parse status, path counts, candidate-set size, relaxation depth, answer-type confidence, GraphLite score margin, and embedding-rank margin.

Remaining execution gap. The combined pipeline would still need a small executor for temporal, numeric, comparative, and superlative filters. The current trie/verifier stack checks graph validity and relation satisfaction, but it does not compute argmax, date overlap, or numeric comparison. DVI filter constraints should therefore compile into deterministic KG-attribute operations before the final answer prompt, so the final LLM receives both evidence paths and checked operator results.

14.1 Expanded Future Work

A direct next step is a learned router. The current exact-size rule is intentionally simple, but it ignores many useful features already saved in DVI metadata. A learned router could predict whether to use GCR, raw DVI, GraphLite-filtered DVI, or an embedding-augmented evidence pool. It should be trained on a validation split and tested on a held-out split to avoid oracle-style leakage.

A second next step is executable filters. The decomposer already distinguishes relation/attribute constraints from filter constraints, but filters are mostly metadata. Temporal, numeric, comparative, and superlative constraints should be compiled into deterministic operations over KG attributes. For example, a governor-in-2009 question should retrieve office intervals and test whether 2009 lies inside the interval.

A third next step is learned answer typing. Current answer-aware refinement uses lightweight heuristics. A stronger version would predict the expected answer type, identify mediator schemas, and expand local graph edges only along relations compatible with the target type. This would directly address failures where the system returns a language instead of a religion or a country instead of a currency.

A fourth next step is unified evidence scoring. Embedding-guided KG-Trie, GraphLite, and DVI currently produce partially separate scores and metadata. A unified evidence object should store path text, entity IDs, source entity, hop length, semantic score, cross-encoder score, relation overlap, generic-source penalty, and constraint ID. This object can be passed through retrieval, verification, intersection, and final answer generation without losing provenance.

15 Conclusion

We reproduced GCR and implemented three modifications for complex KGQA: DVI, GraphLite, and embedding-guided KG-Trie construction. DVI adds explicit constraint decomposition, per-constraint verification, deterministic candidate intersection, and confidence-gated routing. GraphLite studies a faster verifier based on entity-level graph enumeration and neural scoring. Embedding-guided KG-Trie changes trie construction so that semantically relevant 3-hop evidence can enter the decoder without uncontrolled path explosion.

The main empirical result is conservative but useful: exact-size routed DVI slightly improves the strongest reproduced GCR baselines, while oracle routing shows that DVI and GCR have substantial complementarity. GraphLite is faster but currently lower quality. Embedding-guided path selection improves controlled 3-hop probes but does not beat the full 2-hop Qwen baseline by itself. The main research lesson is that KG path validity is only the first layer of faithful reasoning. Complex KGQA needs explicit state for constraint coverage, answer type, entity intersection, and executable temporal, numeric, and superlative operators.

References

[1] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic parsing on Freebase from question-answer pairs. EMNLP.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL.

[3] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. arXiv.

[4] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTa: Decoding-enhanced BERT with disentangled attention. ICLR.

[5] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. 2023. StructGPT: A general framework for large language model to reason over structured data. arXiv.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS.

[7] Linhao Luo et al. 2024. Reasoning on graphs: Faithful and interpretable large language model reasoning. ICLR.

[8] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. 2025. Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models. ICML.

[9] Alexander Miller et al. 2016. Key-value memory networks for directly reading documents. EMNLP.

[10] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP-IJCNLP.

[11] Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen. 2018. Open domain question answering using early fusion of knowledge bases and text. EMNLP.

[12] Haitian Sun, Tania Bedrax-Weiss, and William Cohen. 2019. PullNet: Open domain question answering with iterative retrieval on knowledge bases and text. EMNLP.

[13] Jiashuo Sun et al. 2023. Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph. arXiv.

[14] Alon Talmor and Jonathan Berant. 2018. The Web as a Knowledge-base for Answering Complex Questions. NAACL.

[15] Hugo Touvron et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv.

[16] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. NeurIPS.

[17] Xuezhi Wang et al. 2023. Self-consistency improves chain of thought reasoning in language models. ICLR.

[18] Jason Wei et al. 2022. Chain-of-thought prompting elicits reasoning in language models. NeurIPS.

[19] An Yang et al. 2024. Qwen2 Technical Report. arXiv.

[20] Shunyu Yao et al. 2023. ReAct: Synergizing reasoning and acting in language models. ICLR.

[21] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. ACL.

Individual Contributions

- **kaigeliang**: DVI track, including question decomposition, per-constraint verification, candidate intersection, answer-aware refinement, routed DVI experiments, failure analysis, and final report integration.
- **fangzhehuang**: GraphLite track, including entity-level graph enumeration, relation/entity verifier design, PathScorer integration, runtime comparison, and GraphLite experiment summaries.
- **yaoxianshan**: embedding-guided KG-Trie track, including semantic path ranking, 3-hop path-budget experiments, embedding ablations, artifact cleanup, and poster preparation.

A Appendix: Reproducibility and Extra Details

This appendix contains reproduction commands, schemas, artifact maps, and additional compact notes that support the expanded technical report.

A.1 Dataset Links

The released code does not bundle the full RoG data or Freebase-derived graph files because they are large benchmark artifacts. The submission package instead includes `datasets/DATASET_LINKS.md`. The reproduction commands use the Hugging Face dataset identifiers `rmanluo/RoG-cwq` and `rmanluo/RoG-webqsp`, corresponding to RoG-CWQ and RoG-WebQSP. Local experiments in the archive load these datasets through the same `--data_path rmanluo --d RoG-cwq` and `--data_path rmanluo --d RoG-webqsp` conventions.

A.2 Prompt Templates

This project uses prompts in two places: the KG-specialized model generates graph-valid reasoning paths under a trie, and the general LLM performs decomposition and final answer synthesis. GraphLite is intentionally different: it replaces the KG-LLM prompt with structured graph enumeration and learned scoring inputs. The templates below are the report-facing versions of the code prompts; placeholders such as `{question}` and `{entities}` are filled by the workflow scripts.

Baseline GCR and embedding-guided KG-Trie path prompt. This prompt is used by the baseline path-generation workflow and by the embedding-guided overlay after the path pool has been ranked and admitted into the KG-Trie.

```
Reasoning path is a sequence of triples in the KG that connects the
↪  topic
entities in the question to answer entities. Given a question,
↪  please generate
some reasoning paths in the KG starting from the topic entities to
↪  answer the
question.

# Question:
{question}
# Topic entities:
{entities}
```

DVI decomposition system prompt. The decomposition call asks a general LLM to produce strict JSON rather than prose. This keeps the controller deterministic after the LLM call.

```
You are a knowledge graph query analyzer.
Decompose a complex question into atomic constraints, each anchored
↪  to ONE known
entity. Output valid JSON only. No extra text.
```

DVI decomposition user prompt. The user prompt defines the output schema and gives few-shot examples. The key design decision is that relation and attribute constraints must be anchored to known question entities, while literal constraints become filter predicates.

```
Decompose the question into atomic constraints over the answer
↪  variable.

Use these constraint types:
- "relation":   the answer is reachable from anchor via KG
↪  relations (multi-hop ok)
- "attribute":  the answer has a property/award linking to anchor
- "filter":     the answer must satisfy a literal predicate (date,
↪  type, count,
                nationality)

Rules:
1. Every "relation" or "attribute" constraint must have an "anchor"
↪  from q_entities.
2. Every "filter" constraint must have a "predicate"
↪  (Python-evaluable string when
   possible).
3. Keep constraints minimal -- one constraint per sub-condition.
4. All constraints share the same "answer_variable".

Example output:
{
  "answer_variable": "actor",
  "constraints": [
    {"id": "c1", "type": "relation",  "anchor": "Christopher Nolan",
     "hint": "films directed by Christopher Nolan starring the
     ↪  actor"},
    {"id": "c2", "type": "attribute", "anchor": "Golden Globe",
     "hint": "actor won a Golden Globe award"},
    {"id": "c3", "type": "attribute", "anchor": "Oscar",
     "hint": "actor won an Oscar award"},
    {"id": "c4", "type": "filter", "predicate": "birth_year <
    ↪  1970"},
    {"id": "c5", "type": "filter", "predicate": "nationality ==
    ↪  'American'"}
```

```
  ]
}

Now decompose:
Question: {question}
q_entities: {q_entities}
Output:
```

DVI per-constraint mini-trie prompt. After decomposition, DVI builds a separate trie from each constraint anchor. The text prompt remains close to GCR, but the graph constraint and topic-entity list are now scoped to one atomic condition.

```
Reasoning path is a sequence of triples in the KG that connects the
↪  topic
entities in the question to answer entities. Given a question,
↪  please generate
some reasoning paths in the KG starting from the topic entities to
↪  answer the
question.

# Question:
{question}
# Topic entities:
{constraint_anchor_entities}
```

Final answer prompt with candidates. DVI deliberately tells the final LLM that candidate entities may be intermediate. This was added after failure analysis showed that strict endpoint selection can return mediator nodes or broad type entities.

```
Based on the reasoning paths below, answer the original question.
The candidate entities are KG entities collected from path
↪  endpoints; they may be
intermediate or noisy, so do not restrict your answer to the
↪  candidate list.
Return all possible final answers, one per line. Keep answers
↪  concise (entity
names only).

# Question:
{question}

# Reasoning Paths:
{evidence_paths}

# Candidate or intermediate KG entities:
{candidates}

Answer:
```

Final answer prompt without candidates. This fallback is used when no reliable final candidate set is available but evidence paths still exist.

```
Based on the reasoning paths below, answer the question.
Return all possible answers, one per line. Keep answers concise
↪  (entity names only).

# Question:
{question}

# Reasoning Paths:
{evidence_paths}

Answer:
```

GraphLite scoring inputs. GraphLite does not use a natural-language generation prompt during verification. Instead it converts verification into supervised scoring examples. Relation selection scores `(question, relation)` pairs, and entity verification scores `(question, path, entity)` triples. This is the main architectural difference between GraphLite and KG-LLM DVI: the output space is a ranked list of graph objects rather than generated path text.

A.3 Reproduction Commands

The most important command-line entry points are listed below. They are representative commands rather than a guarantee that every full-test run will finish quickly on a small machine.

```
# Run from the submitted package root.

# Baseline GCR path generation
PYTHONPATH=code/dvi_gcr python
↪  code/dvi_gcr/workflow/predict_paths_and_answers.py \
  --data_path rmanluo --d RoG-cwq --split test \
  --model_name GCR-Meta-Llama-3.1-8B-Instruct \
  --model_path rmanluo/GCR-Meta-Llama-3.1-8B-Instruct \
  --k 10 --index_path_length 2 \
  --prompt_mode zero-shot --generation_mode group-beam

# DVI with KG-LLM verifier
PYTHONPATH=code/dvi_gcr python code/dvi_gcr/workflow/predict_dvi.py
↪  \
  --data_path rmanluo --d RoG-cwq --split test \
  --kg_model_name GCR-Meta-Llama-3.1-8B-Instruct \
  --kg_model_path rmanluo/GCR-Meta-Llama-3.1-8B-Instruct \
  --general_model_name gpt-4o-mini \
  --k 10 --index_path_length 2 \
  --decompose_cache_path <runtime-cache-json> \
  --min_candidates 1

# DVI with PathScorer verifier
PYTHONPATH=code/dvi_gcr python code/dvi_gcr/workflow/predict_dvi.py
↪  \
  --data_path rmanluo --d RoG-cwq --split test \
  --general_model_name gpt-4o-mini \
  --index_path_length 2 \
  --path_scorer --bi_k 100 --cross_k 10
```

A.4 Prediction Record Schema

A DVI prediction record is a JSON object with a final answer plus intermediate fields. The exact schema can evolve, but the fields used in this report are conceptually:

- `id`, `question`, `prediction`, and optional gold-answer fields.
- `decomposition`: answer variable, relation constraints, attribute constraints, and filters.
- `paths_per_constraint`: accepted or ranked evidence paths for each constraint.
- `raw_final_candidates` and `final_candidates`: candidates before and after answer-aware refinement.
- `intersect_stats`: strict size, relaxed size, dropped constraints, and candidate-set diagnostics.
- `timing`: decomposition, KG decoding, path scoring, intersection, refinement, final answer, and total time.

A.5 Detailed Artifact Map

A.6 Code Locations

- DVI implementation: `code/dvi_gcr/workflow/predict_dvi.py`, `code/dvi_gcr/src/dvi/decomposer.py`, `code/dvi_gcr/src/dvi/intersector.py`, `code/dvi_gcr/src/dvi/answer_aware.py`, and `code/dvi_gcr/src/qa_prompt_builder.py`.
- GraphLite entity-level prototype: `code/graphlite/lite_framework.rar`; extracted source files are packaged at `code/graphlite/lite_framework/entity_level_decoder.py`, `code/graphlite/lite_framework/cross_encoder_verifier.py`, `code/graphlite/lite_framework/graph_walker.py`, `code/graphlite/lite_framework/graph_encoder.py`, `code/graphlite/lite_framework/train_cross_encoder.py`, and `code/graphlite/lite_framework/train_entity_verifier.py`.
- Integrated GraphLite / PathScorer implementation: `code/dvi_gcr/src/dvi/path_scorer.py`, `code/dvi_`

Artifact	Description
code/dvi_gcr/workflow/predict_dvi.py	End-to-end DVI pipeline with KG-LLM and PathScorer verification modes.
code/dvi_gcr/src/dvi/decomposer.py	General-LLM JSON decomposer with fallback and cache support.
code/dvi_gcr/src/dvi/intersector.py	Tail-entity extraction, candidate-set construction, strict intersection, and relaxed diagnostics.
code/dvi_gcr/src/dvi/answer_aware.py	Heuristic answer-type refinement and local graph expansion.
code/dvi_gcr/src/dvi/path_scorer.py	Bi-encoder filtering and cross-encoder reranking for enumerated paths.
code/embedding_guided_kgtrie/overlay/src/qa_prompt_builder.py	Embedding-guided path ranking and KG-Trie construction overlay.
code/graphlite/lite_framework/entity_level_decoder.py	Standalone GraphLite entity-level graph enumeration and scoring prototype.
results_summary/EXPERIMENT_TABLES.md	GitHub-readable experiment tables corresponding to the report.

Table 10: Expanded artifact map for reproducing and extending the project.

gcr/workflow/build_path_verifier_data.py, and code/dvi_gcr/workflow/train_path_reranker.py.

- Embedding-guided KG-Trie implementation: code/embedding_guided_kgtrie/overlay/src/qa_prompt_builder.py and code/embedding_guided_kgtrie/overlay/workflow/predict_paths_and_answers.py.
- Packaged result summaries: results_summary/EXPERIMENT_TABLES.md and results_summary/RESULTS.md. These files contain the full-test baseline, raw DVI, routed DVI, GraphLite/PathScorer, embedding-guided KG-Trie, oracle-routing, and failure-analysis tables used in the report.

A.7 DVI Gate Ablation

Model	Gate	Acc./Rec.	Hit@1	F1	Time(s)
Llama-3.1	$\|C\| \leq 1$	59.03	54.80	54.33	8.54
Llama-3.1	$\|C\| = 2$	59.26	56.22	54.63	6.58
Llama-3.1	$\|C\| \leq 2$	59.28	55.14	54.41	10.29
Llama-3.1	$\|C\| \leq 3$	58.92	54.63	53.87	11.27
Llama-2	$\|C\| \leq 1$	60.40	55.88	54.38	6.89
Llama-2	$\|C\| = 2$	60.56	56.92	54.51	6.53
Llama-2	$\|C\| \leq 2$	60.45	55.68	54.40	7.16
Llama-2	$\|C\| \leq 3$	59.71	54.91	53.84	7.34

Table 11: DVI confidence-gate ablation. Exact-size routing is the most stable setting in this archive.

A.8 GraphLite Training Signal

The trained cross-encoder path reranker improves held-out grouped path retrieval before QA: Hit@1 rises from 56.33% to 69.62%, MRR from 0.680 to 0.771, and Hit@10 from 88.61% to 91.77%. A 100-example QA probe with the trained scorer reaches 50.55 F1, and a post-hoc evidence setting reaches 53.52 F1. This improves over random or untrained path selection but remains below full KG-LLM DVI and strong GCR baselines.

A.9 Additional Embedding Ablations

On CWQ20, the 2-hop group-beam baseline has 27.83 F1, 2-hop beam early stopping has 30.50 F1, naive 3-hop with a path budget has 22.37 F1, and 3-hop embedding-guided top-1 reaches 45.56 F1. Hybrid lexical and structural scoring matches pure embedding in the best small setting but does not clearly exceed it. The main practical lesson is to protect short-hop coverage and use semantic ranking only to add evidence, not to prune the entire graph neighborhood aggressively.

A.10 Failure Case Details

College question. Question: "Where did the Country Nation World Tour concert artist go to college?" Gold: Belmont University. The baseline often follows generic paths from *College/University*; embedding sometimes retrieves more related paths but still predicts broad entities such as United States of America. A correct system must identify the concert artist and then follow the education relation to the institution.

Afghan National Anthem question. Question: "The national anthem Afghan National Anthem is from the country which practices what religions?" Gold: Sunni Islam and Shia Islam. Generated paths often follow music-language relations and return Pashto language. This is a relation-mismatch failure: the path is related to the anthem but not to the asked religion predicate.

Libya anthem question. Question: "Which man is the leader of the country that uses Libya, Libya, Libya as its national anthem?" Gold: Abdullah al-Thani. The system retrieves paths to Libya and prime-minister entities but returns generic roles or languages. This is a multi-entity intersection plus answer-type failure.

A.11 Packaged Result Files

- Main experiment tables and ablations: results_summary/EXPERIMENT_TABLES.md.
- Compact metric summary: results_summary/RESULTS.md.
- Dataset download and loading links: datasets/DATASET_LINKS.md.

The raw generated prediction directories are intentionally not part of the submission package because they are large and re-

generable. All file paths named in the artifact map and code-location list above are relative to the submitted package root.